Finançat per la Unió Europea

IES MESTRE RAMON ESTEVE

GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura, Universitats i Ocupació

UP4.6. REPOSITORIO GITHUB

DESPLIEGUE DE APLICACIONES WEB

2º CFGS – DESARROLLO DE APLICACIONES WEB

RA6. Elabora la documentación de la aplicación web evaluando y seleccionando herramientas de generación de documentación, control de versiones y de integración continua.

PABLO PÉREZ GONZÁLEZ

CURSO 2025/26

ÍNDICE

1. **REPOSITORIO GITHUB.**
   - DEFINICIÓN.
   - CONCEPTOS CLAVE.
2. **CREACIÓN DEL REPOSITORIO GITHUB.**
   - PROCESO DE CREACIÓN DE REPOSITORIO REMOTO.
3. **USO DEL REPOSITORIO DE GITHUB.**
   - ACERCA DE REPOSITORIOS REMOTOS.
   - GESTIÓN DE TOKENS DE ACCESO PERSONALES.
   - ENVÍO DE COMMITS A UN REPOSITORIO REMOTO.
   - OBTENCIÓN DE CAMBIOS DESDE UN REPOSITORIO REMOTO.
4. **TRABAJANDO CON GITHUB EN VS CODE.**
   - INTRODUCCIÓN.
   - VINCULAR GITHUB CON VISUAL STUDIO CODE.
   - CONFLICTOS.
   - EJEMPLOS DE CONFLICTOS.

RESULTADO DE APRENDIZAJE 6

CRITERIO DE EVALUACIÓN h)

RA6. Elabora la documentació de l'aplicació web avaluant i seleccionant ferramentes de generació de documentació, control de versions i d'integració contínua.	
h) S'han utilitzat ferramentes per a la integració contínua del codi.	15%

REPOSITORIO GITHUB

DEFINICIÓN

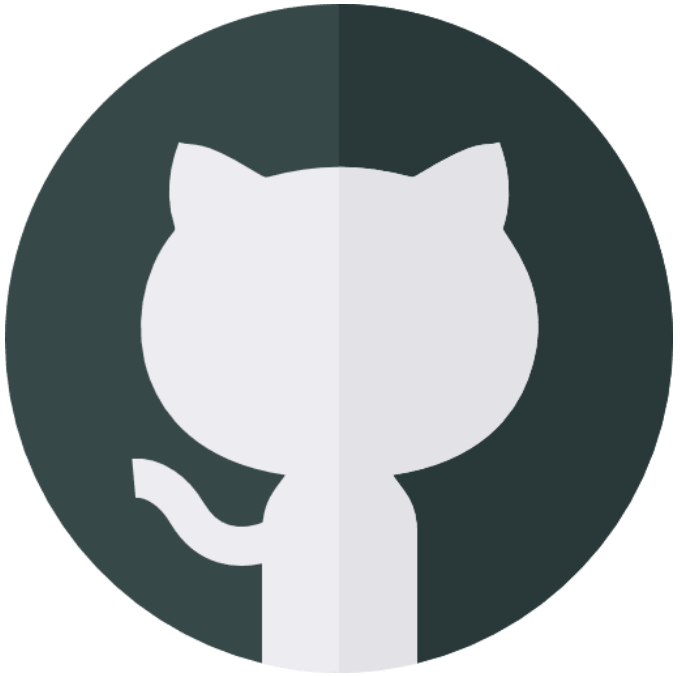
GitHub es la plataforma en línea que te permite gestionar proyectos mediante el control de versiones de Git (Sistemas de control de versiones distribuidos, DVCS). Un **repositorio** es el elemento más básico de GitHub. Es un lugar donde puedes almacenar tu código, tus archivos y el historial de revisiones de cada archivo.

Como repositorio, cualquier usuario puede descargar y revisar el código de los proyectos de código abierto, lo que ayuda a mejorar el producto y a crear ramas a partir de él. Y si prefieres que tu código no se vea, también puedes crear proyectos privados.

REPOSITORIO GITHUB

CONCEPTOS CLAVE

Term	Definition
Branch	A parallel version of your code that is contained within the repository, but does not affect the primary or main branch.
Clone	To download a full copy of a repository's data from GitHub.com, including all versions of every file and folder.
Fork	A new repository that shares code and visibility settings with the original "upstream" repository.
Merge	To take the changes from one branch and apply them to another.
Pull request	A request to merge changes from one branch into another.
Remote	A repository stored on GitHub, not on your computer.
Upstream	The branch on an original repository that has been forked or cloned. The corresponding branch on the cloned or forked branch is called the "downstream."

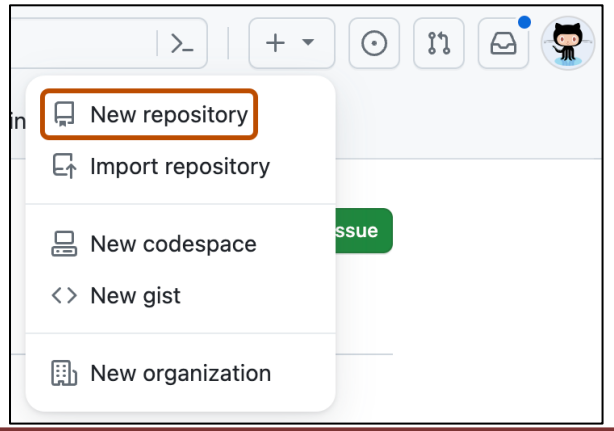


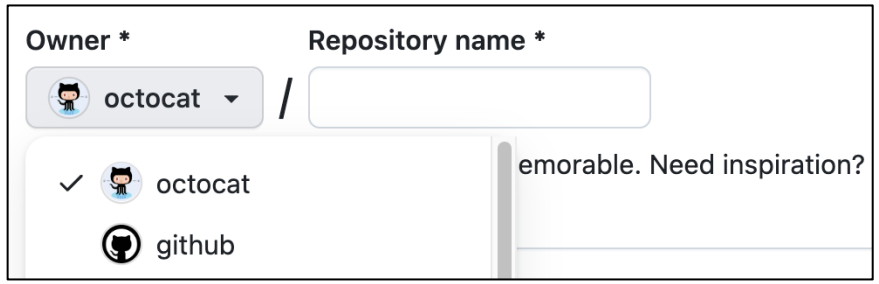
CREACIÓN DEL REPOSITORIO GITHUB

PROCESO DE CREACIÓN DE REPOSITORIO REMOTO

Para **crear un nuevo repositorio de GitHub**, accede a Creating a new repository.

- Puedes añadir una descripción de tu repositorio.
- Este repositorio puede ser público (cualquier persona en internet puede verlo) o privado (tú eliges quién puede verlo y confirmarlo).
- Puedes crear un README, que es un documento que describe tu proyecto.
- Puedes crear un `.gitignore` file, que es un conjunto de reglas de omisión.
- Puedes optar por agregar una licencia software para tu proyecto.

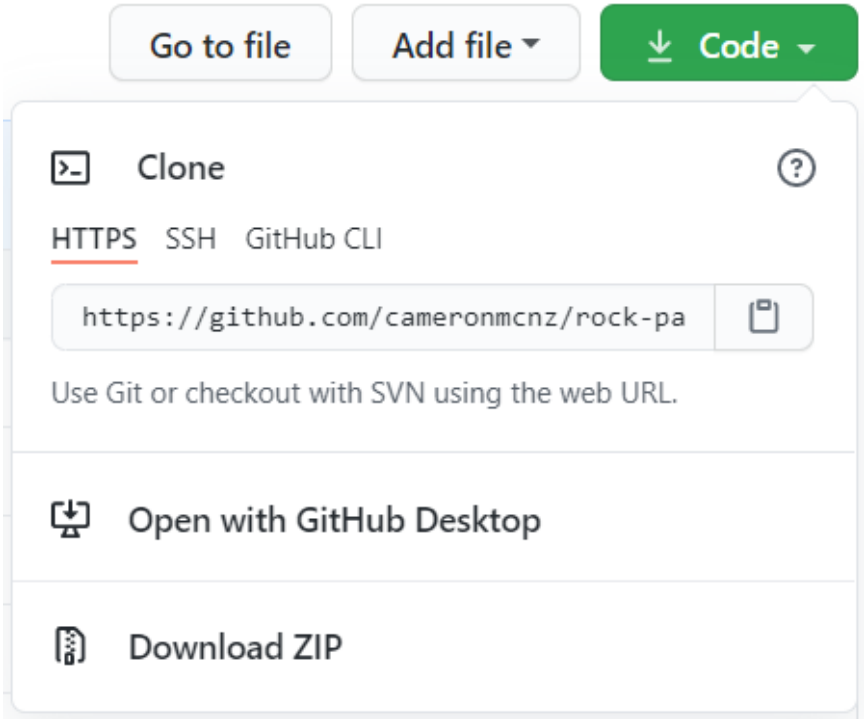


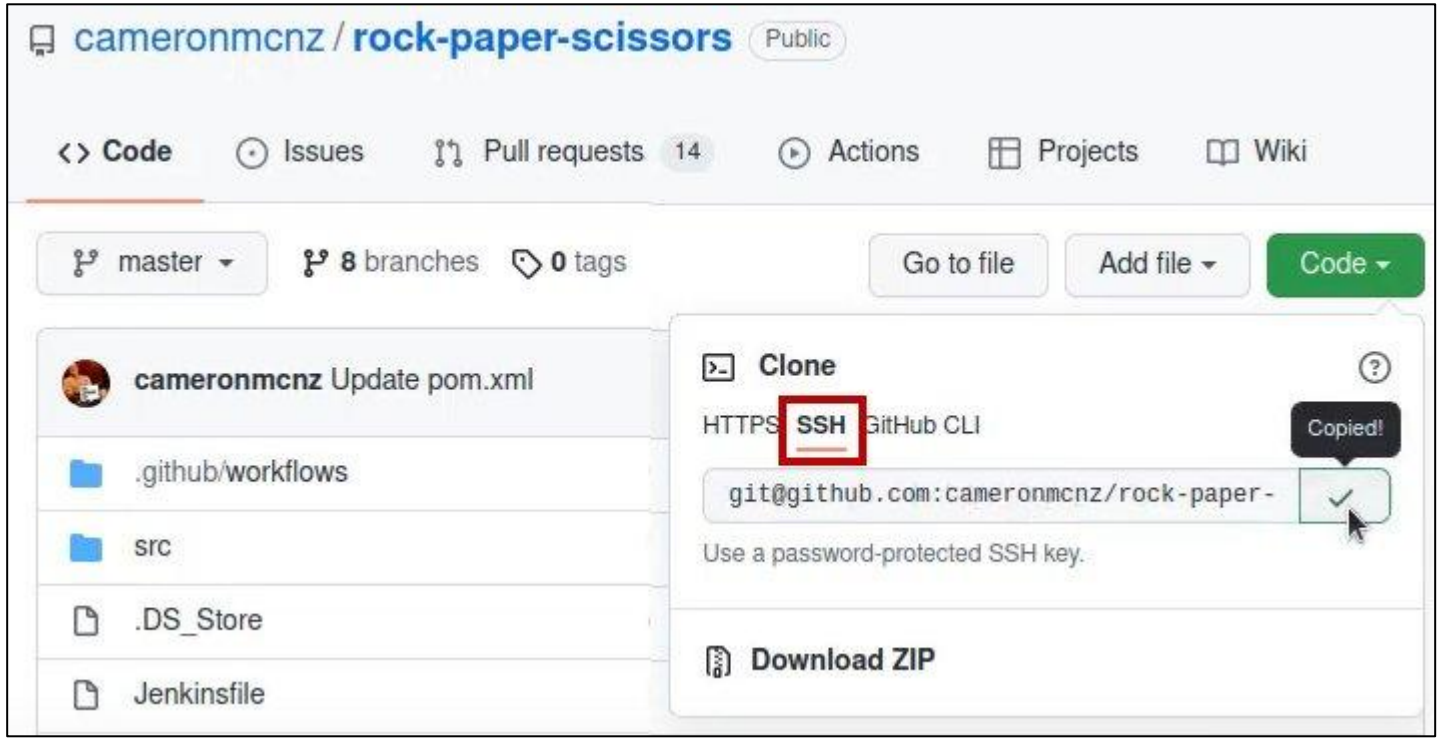


USO DEL REPOSITORIO DE GITHUB

ACERCA DE REPOSITORIOS REMOTOS

El **enfoque colaborativo de GitHub** para el desarrollo se basa en la publicación de commits desde tu repositorio local en GitHub para que otras personas puedan verlos, recuperarlos y actualizarlos. Puedes acceder a la página About remote repositories para obtener más información.



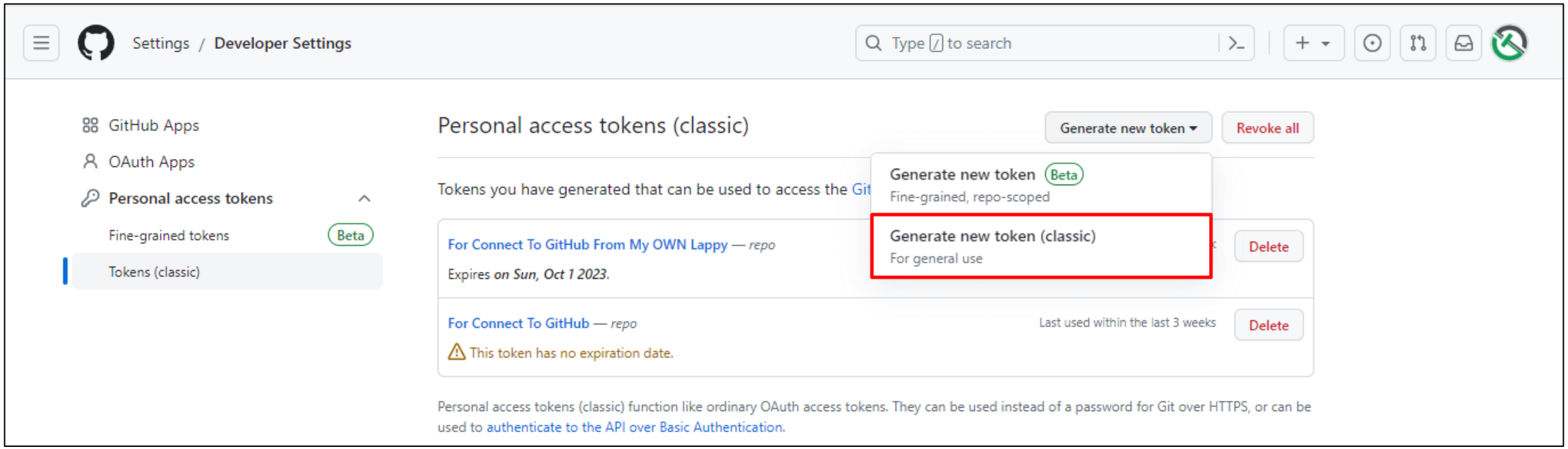


USO DEL REPOSITORIO DE GITHUB

GESTIÓN DE TOKENS DE ACCESO PERSONALES

Puedes usar un **token de acceso personal** en lugar de una contraseña al autenticarte en GitHub en la línea de comandos o con la API.

Puedes acceder a la página Managing your personal access tokens para obtener más información.

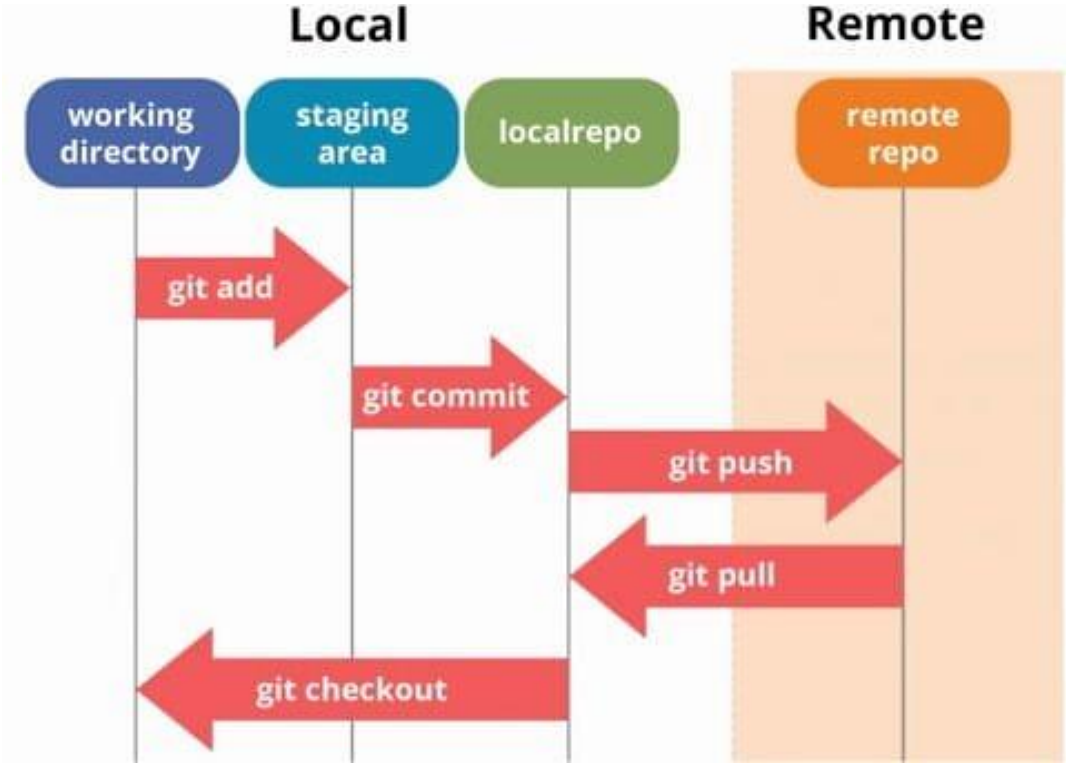


USO DEL REPOSITORIO DE GITHUB

ENVÍO DE COMMITS A UN REPOSITORIO REMOTO

Usa **`git push`** para enviar commits realizados en tu rama local a un repositorio remoto.

Puedes acceder a la página Pushing commits to a remote repository para obtener más información.

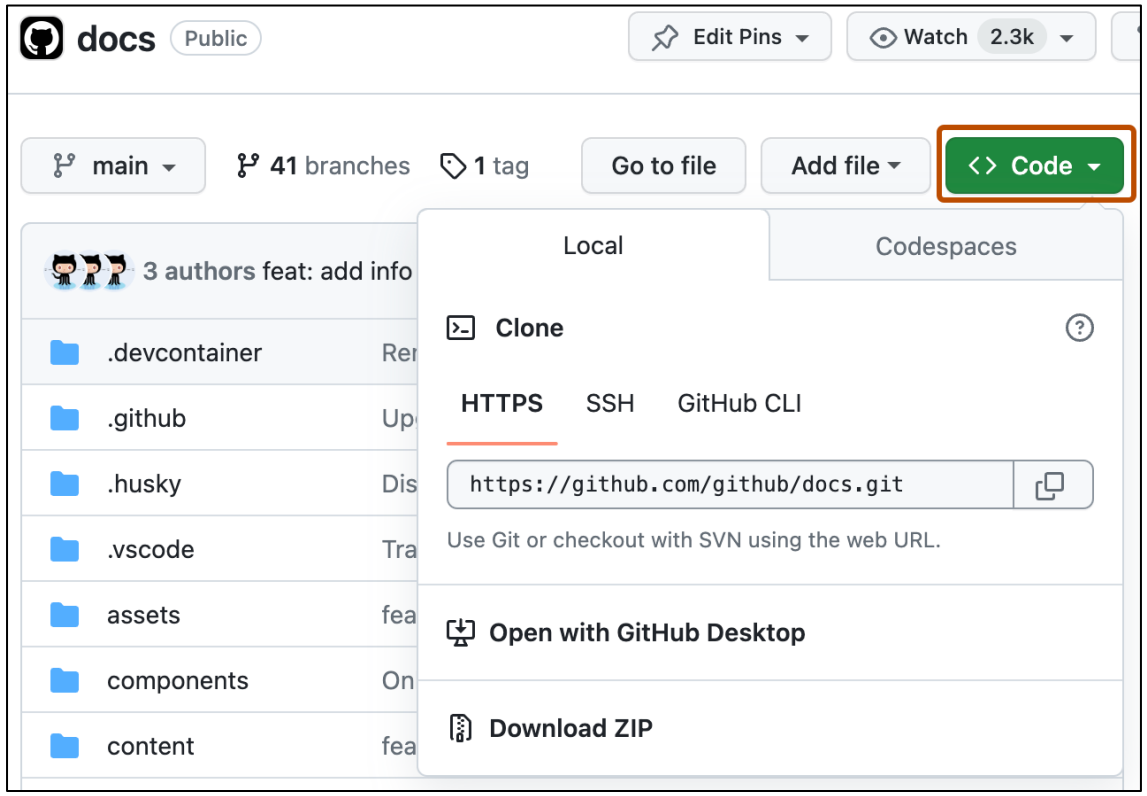


USO DEL REPOSITORIO DE GITHUB

OBTENCIÓN DE CAMBIOS DESDE UN REPOSITORIO REMOTO

Puedes usar **comandos comunes de Git** para acceder a repositorios remotos.

Puedes acceder a la página Getting changes from a remote repository para obtener más información.



TRABAJANDO CON GITHUB EN VS CODE

INTRODUCCIÓN

Usar **GitHub con Visual Studio Code** te permite compartir tu código fuente y colaborar con otros directamente desde tu editor.

Hay muchas maneras de interactuar con GitHub, por ejemplo, a través de su sitio web https://github.com o la interfaz de línea de comandos de Git. En VS Code, la integración de GitHub es proporcionada por la extensión GitHub Pull Requests and Issues.

Puedes acceder a la página Working with GitHub in VS Code para obtener más información.

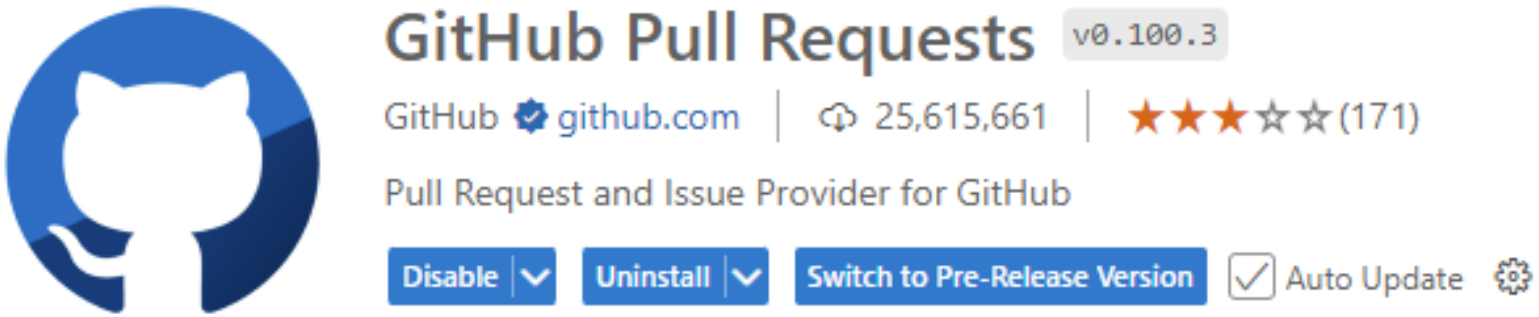
TRABAJANDO CON GITHUB EN VS CODE

VINCULAR GITHUB CON VISUAL STUDIO CODE

Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos:

Paso 1: Instalar la extensión de GitHub en VS Code.

- Abre VS Code.
- Ve a la sección Extensiones.
- Busca "*GitHub Pull Requests*" e instálalo.
- Asegúrate también de que la extensión de Git esté instalada y configurada (nombre de usuario y correo electrónico).



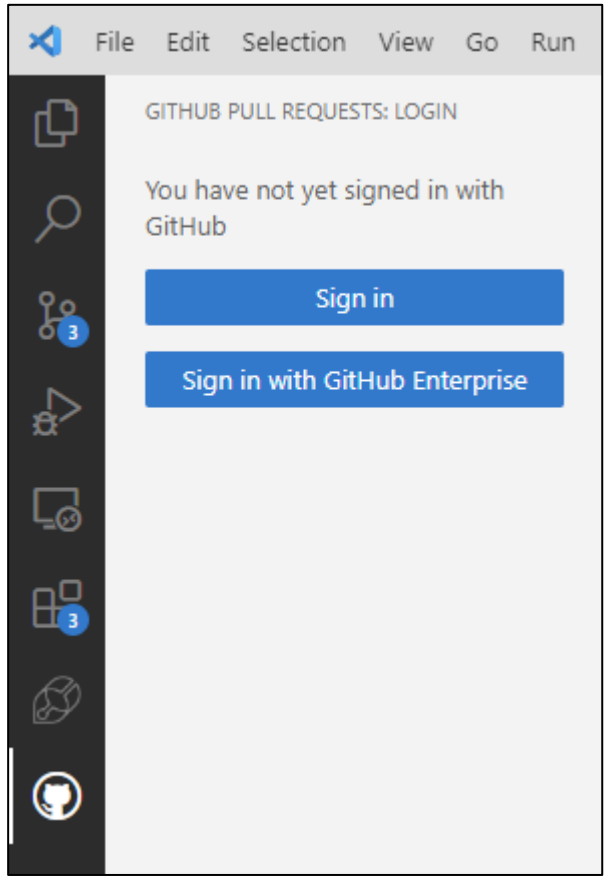
TRABAJANDO CON GITHUB EN VS CODE

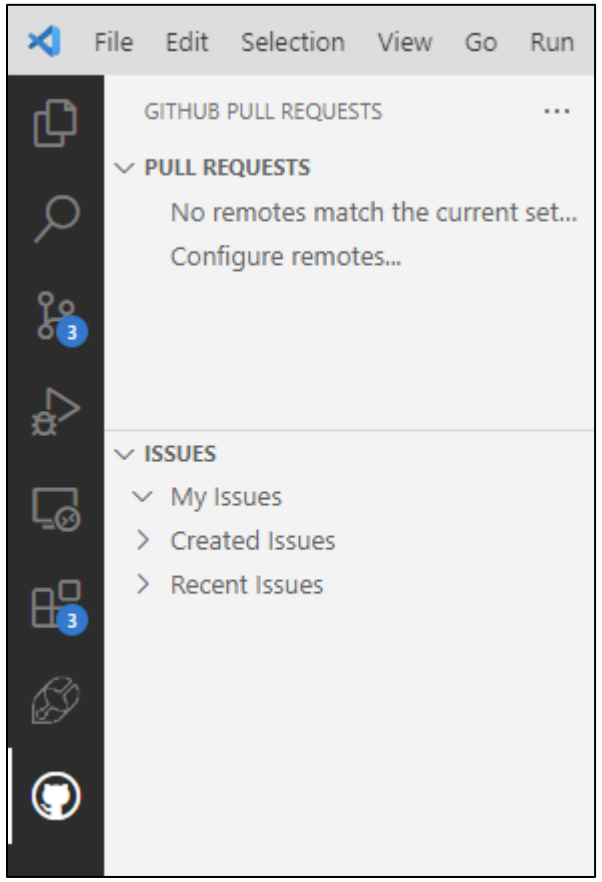
VINCULAR GITHUB CON VISUAL STUDIO CODE

Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos:

Paso 2: Inicia sesión en GitHub desde VS Code

- Haz clic en el ícono de tu cuenta de GitHub.
- Selecciona Iniciar sesión.
- Aparecerá un enlace donde deberás iniciar sesión en GitHub y autorizar a VS Code a acceder a tu cuenta.
- Una vez autorizado, regresa a VS Code y confirma la conexión.





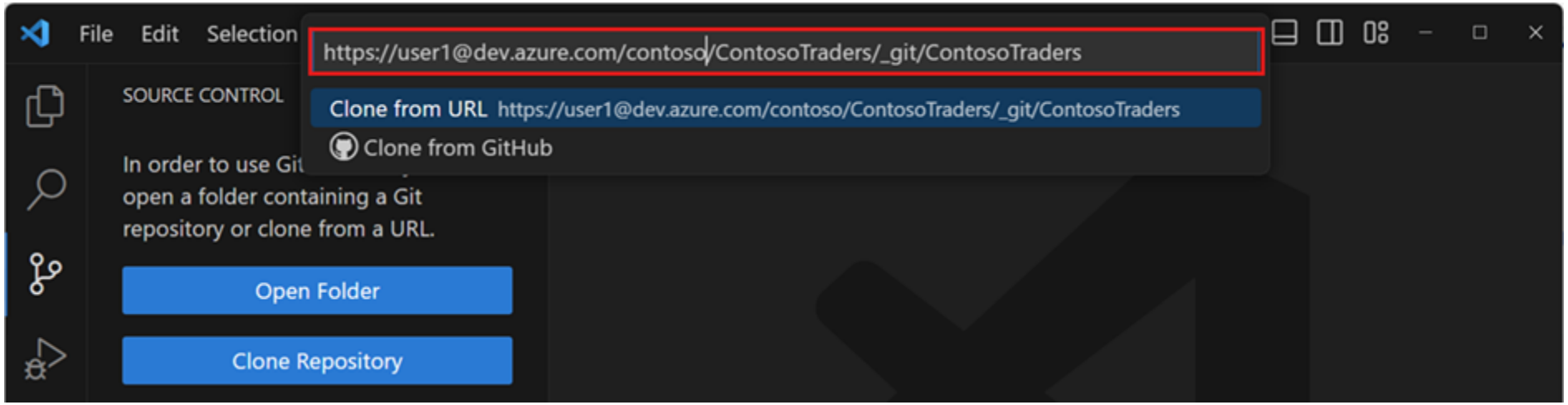
TRABAJANDO CON GITHUB EN VS CODE

VINCULAR GITHUB CON VISUAL STUDIO CODE

Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos:

Paso 3: Clona un repositorio existente (si ya tienes un repositorio en GitHub):

- Haz click en *View* > *Command Palette* (Ctrl+Shift+P), escribe “*Git: Clone*” y selecciónalo.
- Puedes hacer click en "*Clone Repository*" en lugar del paso anterior.
- Introduce la URL de tu repositorio de GitHub.



TRABAJANDO CON GITHUB EN VS CODE

VINCULAR GITHUB CON VISUAL STUDIO CODE

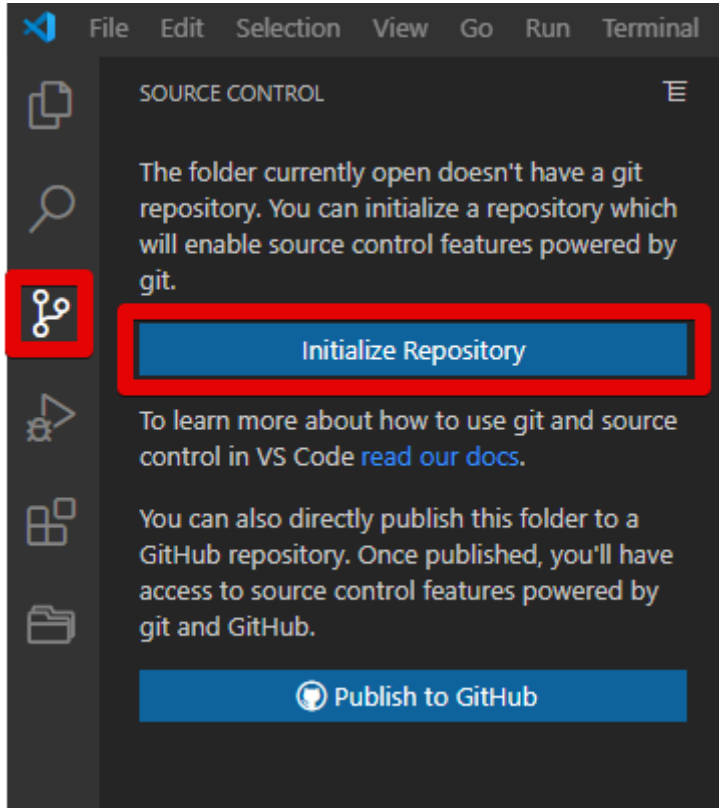
Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos:

Paso 4: Crea un nuevo repositorio desde VS Code (si no tienes un repositorio, puedes crear uno directamente desde VS Code):

- Abre la carpeta de tu proyecto en VS Code.
- Ve a la vista Control de código fuente y haz clic en "*Initialize Repository*".
- Ve a la terminal y conecta tu repositorio a GitHub:

```
git remote add origin
https://github.com/TuUsuario/TuRepositorio.git
git branch -M main
git push -u origin main
```

- Esto creará un repositorio remoto y enviará su código.



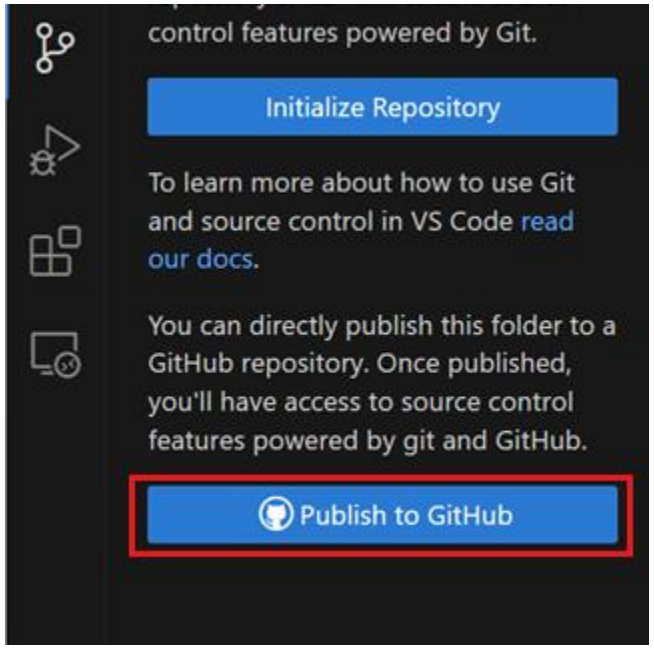
TRABAJANDO CON GITHUB EN VS CODE

VINCULAR GITHUB CON VISUAL STUDIO CODE

Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos:

Paso 5: Publica tu repositorio desde VS Code (si tienes un repositorio local):

- Ves a la vista *Source Control* y haz click en "*Publish to GitHub*".
- Puede elegir un nombre y una descripción del repositorio y hacerlo público o privado.



TRABAJANDO CON GITHUB EN VS CODE

VINCULAR GITHUB CON VISUAL STUDIO CODE

Para vincular GitHub con Visual Studio Code, debemos seguir estos pasos :

Paso 6: Trabajar con Git en VS Code

Realizar commits:

- En la vista de *Source Control*, verás los archivos modificados.
- Agrega un mensaje descriptivo en la barra superior y haz clic en el ícono de verificación para realizar el commit.

Sincronizar cambios: Haz click en el icono de sincronización para cargar los cambios en GitHub.

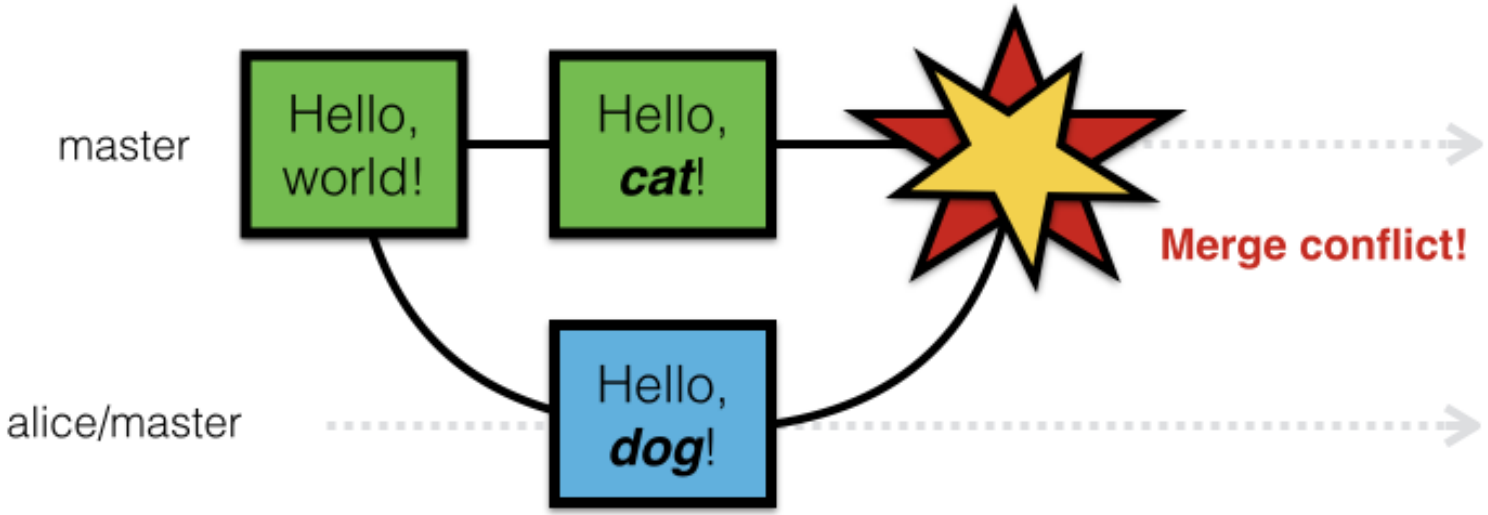
Realizando Pull y Push:

- Utiliza la terminal o los botones en la vista de *Source Control* para administrar los cambios entre tu repositorio local y remoto:
  - Pull: `git pull origin main` (descargar cambios).
  - Push: `git push origin main` (cargar cambios).

¿Qué es un conflicto en Git?

Un conflict ocurre cuando:

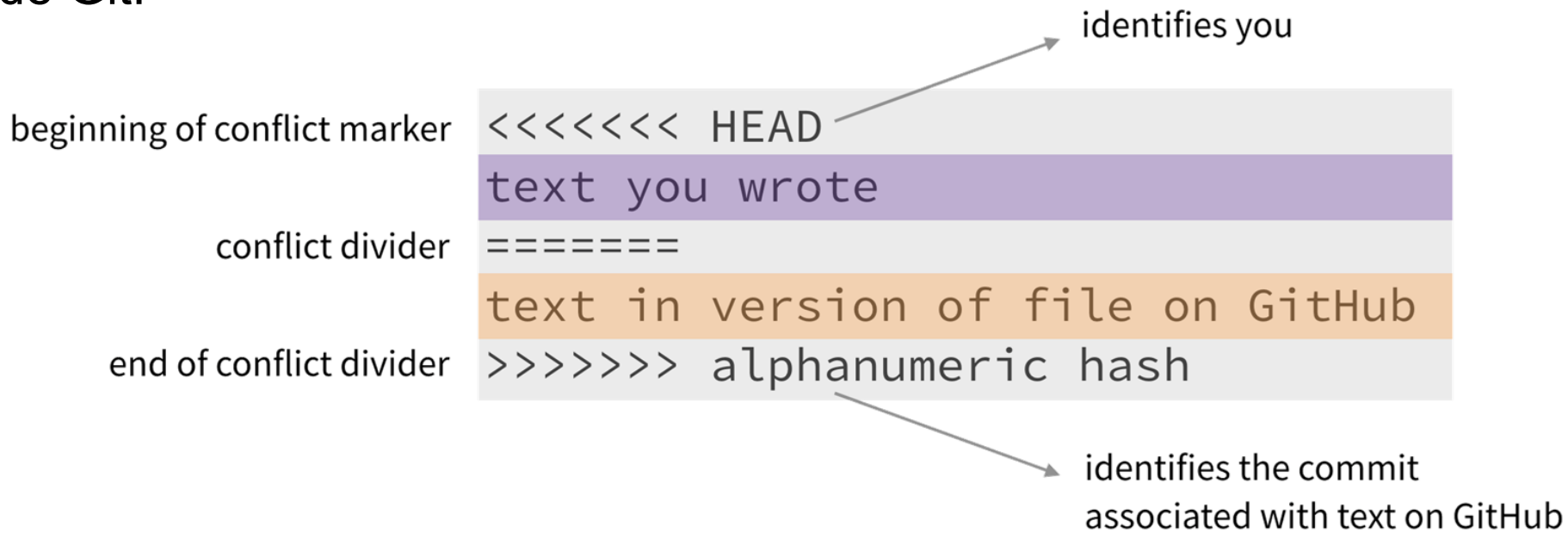
- Dos personas editan el mismo archivo en la misma línea de código.
- Intentas hacer **merge** o **pull**, pero Git no puede fusionar automáticamente los cambios.



TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

Cuando abres un archivo con conflictos, verás el código rodeado por los indicadores de conflicto de Git:



VS Code reconoce los conflictos de merge. Las diferencias se resaltan y existen acciones en línea para aceptar uno o ambos cambios. Una vez resueltos los conflictos, debes preparar el archivo conflictivo para poder confirmar los cambios.

TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

```
14  /**
15   * Prints the welcome message
16  */
    Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
17  <<<<<<< HEAD (Current Change)
18  function printMessage(showUsage, message) {
19      console.log(message);
20
21  =======
22  function printMessage(showUsage, showVersion) {
23      console.log("Welcome To Line Counter");
24      if (showVersion) {
25          console.log("Version: 1.0.0");
26      }
27  >>>>>>> theirs (Incoming Change)
28      if (showUsage) {
29          console.log("Usage: node base.js <file1> <file2> ... ");
30      }
31  }
32
33  /**
```

Resolve in Merge Editor

You, 20 seconds ago Ln 11, Col 26 Spaces: 4 UTF-8 CRLF {} JavaScript

TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

Para ayudarte a resolver conflictos de merge, VS Code ofrece un **editor 3-way merge** donde puedes aceptar interactivamente los cambios entrantes y actuales, y ver y editar el archivo fusionado resultante. El editor 3-way merge se abre seleccionando el botón "*Resolve in Merge Editor*" en la esquina inferior derecha de un archivo con conflictos de fusión de Git.

El editor 3-way merge muestra vistas separadas para:

Incoming changes (a la izquierda) → Representa los cambios de código que estás extrayendo de la rama `main` o las modificaciones realizadas por otros desarrolladores.

Current changes (a la derecha) → Representa los cambios de código realizados en tu rama local.

Result of the merge (en la parte inferior) → Decisiones finales sobre los cambios.

TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

target.js ! | Merging: target.js !

merge-git-playground > target.js > printMessage

Incoming 7b18bdb • theirs

```
13
14  /**
15   * Prints the welcome message
16   */
    Accept Incoming | Accept Combination (Incoming First) | Ignore
17  function printMessage(showUsage, showVersion) {
18      console.log("Welcome To Line Counter");
19      if (showVersion) {
20          console.log("Version: 1.0.0");
21      }
22      if (showUsage) {
23          console.log("Usage: node base.js <file1>
24      }
```

Current b7bd9b1 • main

```
13
14  /**
15   * Prints the welcome message
16   */
    Accept Current | Accept Combination (Current First) | Ignore
17  function printMessage(showUsage, message) {
18      console.log(message);
19
20      if (showUsage) {
21          console.log("Usage: node base.js <file1>
22      }
```

Result merge-git-playground\target.js — 1 Conflict Remaining

```
13
14  /**
15   * Prints the welcome message
16   */
    No Changes Accepted
17  function printMessage(showUsage) {
18      console.log("Welcome To Line Counter");
19
20      if (showUsage) {
```

Complete Merge

TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

Para obtener más información sobre cómo funciona el **editor 3-way merge**:

TRABAJANDO CON GITHUB EN VS CODE

CONFLICTOS

Consejos para evitar conflictos:

- **Pull frecuentes:** Antes de empezar a trabajar, sincroniza tu rama local con la remota.
- **Comunicación en equipo:** Coordínate con otros desarrolladores para evitar modificar los mismos archivos.
- **Ramas separadas:** Trabaja en diferentes ramas y fusiona los cambios cuando estén listos.
- **Resuelve conflictos lo más pronto posible:** Gestiona pequeños conflictos para evitar que se acumulen.

EJEMPLOS DE CONFLICTOS

Ejemplo 1: Cambio en la misma línea

Local:

```
function greeting() {
  echo "Hola, mundo!";
}
```

Remote:

```
function greeting() {
  echo "Hello, world!";
}
```

Resulting conflict:

```
<<<<<<< HEAD
function greeting() {
  echo "Hola, mundo!";
}
=======
function greeting() {
  echo "Hello, world!";
}
>>>>>>> origin/main
```

Combined resolution:

```
function greeting() {
  echo "Hola, mundo! Hello, world!";
}
```

EJEMPLOS DE CONFLICTOS

Ejemplo 2: Cambio en líneas diferentes

Local:

```
function sum($a, $b) {
  return $a + $b;
}
function sub($a, $b) {
  return $a - $b;
}
```

Remote:

```
function sum($a, $b) {
  return $a + $b;
}
function mult($a, $b) {
  return $a * $b;
}
```

Resulting conflict:

Git can automatically merge because there are no changes on the same line.

Automatic resolution:

```
function sum($a, $b) {
  return $a + $b;
}
function sub($a, $b) {
  return $a - $b;
}
function mult($a, $b) {
  return $a * $b;
}
```

Ejemplo 3: Cambios incompatibles en la lógica

Local:

```
function checkUser($user) {
 if ($user === "admin") {
  echo "Access granted";
 } else {
  echo "Access denied";
 }
}
```

Remote:

```
function checkUser($user) {
 if ($user === "root") {
  echo "Welcome, root";
 } else {
  echo "Permission denied";
 }
}
```

Combined resolution:

```
function verifiesUser($user) {
 if ($user === "admin" || $user ===
"root") {
  echo "Access granted";
 } else {
  echo "Access denied";
 }
}
```

Ejemplo 4: Cambio de estructura de la función

Local:

```
function calculate($a, $b, $operator) {
 if ($operator === "sum") {
  return $a + $b;
 }
 return 0;
}
```

Remote:

```
function calculate($a, $b, $operator) {
 if ($operator === "multiplication") {
  return $a * $b;
 }
 return 0;
}
```

Combined resolution:

```
function calculate($a, $b, $operator) {
 if ($operator === "sum") {
  return $a + $b;
 }       elseif       ($operator       ===
"multiplication") {
  return $a * $b;
 }
 return 0;
}
```

Ejemplo 5: Cambio archivo de configuración

Local (*config.php*):

```
return [
  'db_host' => 'localhost’,
  'db_user' => 'root’,
  'db_pass' => 'local_password’,
  'db_name' => 'my_local_base’
];
```

Remote:

```
return [
  'db_host' => 'localhost’,
  'db_user' => 'root’,
  'db_pass' => 'remote_password’,
  'db_name' => 'my_remote_base'
];
```

Combined resolution:

```
return [
  'db_host' => 'localhost’,
  'db_user' => 'root’,
  'db_pass'  =>  getenv('DB_PASSWORD')  ?:
'default_password’,
  'db_name'   =>   getenv('DB_NAME')   ?:
'my_base’
];
```

Note: Using environment variables is a good practice to avoid conflicts and protect sensitive information.

Ejemplo 6: Clase modificada por dos personas

Local:

```
class User {
 public function getName() {
  return "Local User";
 }
}
```

Remote:

```
class User {
 public function getName() {
  return "Remote User";
 }
}
```

Combined resolution:

```
class User {
 public function getName($type = 'local') {
  if ($type === 'local') {
   return "Local User";
  }
  return "Remote User";
 }
}
```

Ejemplo 7: Conflicto en múltiples lugares

Local:

```
function getData() {
  $data = ["one", "two", "three"];
  return $data;
}
function processData($data) {
  return array_map('strtoupper', $data);
}
```

Remote:

```
function getData() {
  $data = ["four", "five", "six"];
  return $data;
}
function processData($data) {
  return array_map('strtolower', $data);
}
```

Combined resolution:

```
function getData($type = 'local') {
 if ($type === 'local') {
  $data = ["one", "two", "three"];
 } else {
  $data = ["four", "five", "six"];
 }
 return $data;
}

function   processData($data,   $mode   =
'uppercase') {
 if ($mode === 'uppercase') {
  return array_map('strtoupper', $data);
 }
 return array_map('strtolower', $data);
}
```

PREGUNTAS

DUDAS Y SUGERENCIAS